



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Сканирование портов — это процесс проверки **TCP** или **UDP** портов на удаленной машине с целью обнаружения того, какие службы запущены на объекте и какие потенциальные векторы атак могут существовать.

Важно понимать последствия сканирования портов, а также влияние, которое может оказать сканирование конкретных портов. Из-за объема трафика, который могут генерировать некоторые виды сканирования, а также их навязчивого характера, выполнение сканирования портов вслепую может иметь негативные последствия для целевых систем или сети клиента, например, перегрузить серверы и сетевые каналы или вызвать срабатывание [IDS](#) – системы обнаружения вторжений.

Использование правильной методологии сканирования портов может значительно повысить нашу эффективность как тестеров на проникновение, одновременно ограничивая многие риски. В зависимости от масштаба задания, вместо полного сканирования портов целевой сети мы можем начать со сканирования только портов **80** и **443**. Имея список возможных веб-серверов, мы можем запустить полное сканирование портов этих серверов в фоновом режиме, одновременно выполняя другие действия. После завершения полного сканирования портов мы можем еще больше сузить область сканирования, чтобы получать все больше и больше информации при каждом последующем сканировании. Сканирование портов следует рассматривать как динамический процесс, который уникален для каждого случая. Результаты одного сканирования определяют тип и объем следующего сканирования.

Сканирование TCP/UDP

Мы начнем изучение сканирования портов с простого сканирования портов TCP и UDP с помощью Netcat. Следует отметить, что Netcat не является сканером портов, но он может быть использован в качестве такового в зачаточном виде. Поскольку он уже присутствует во многих системах, мы можем использовать некоторые его функции для имитации базового сканирования портов, когда нам не нужен полнофункциональный сканер портов. Однако существуют гораздо более совершенные инструменты, предназначенные для сканирования портов, которые мы также подробно рассмотрим.

Сканирование TCP

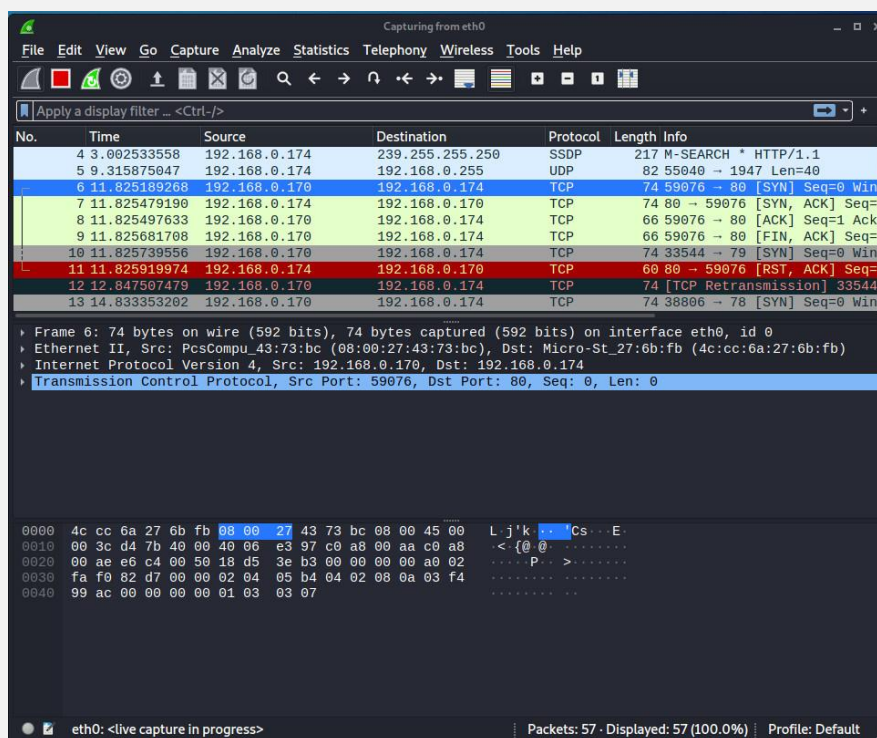
Простейшая техника сканирования портов TCP, обычно называемая сканированием CONNECT, опирается на трехсторонний механизм TCP handshake. Этот механизм разработан таким образом, чтобы два узла, пытающиеся установить связь, могли согласовать параметры сетевого соединения перед передачей каких-либо данных. Сначала хост посылает TCP SYN пакет серверу на порт назначения. Если порт назначения открыт, сервер отвечает пакетом SYN-ACK, а клиентский хост посылает пакет ACK для завершения "рукопожатия". Если рукопожатие завершается успешно, порт считается открытым.

Чтобы продемонстрировать это, мы запустим сканирование портов на портах 75-80. Опция -w задает таймаут соединения в секундах, а -z используется для указания режима zero-I/O, в котором не будут отправляться никаких данных и который используется для сканирования:

```
kali@kali:~$ nc -nvv -w 1 -z 192.168.0.174 75-80
```

```
(kali@kali)-[~]  
$ nc -nvv -w 1 -z 192.168.0.174 75-80  
(UNKNOWN) [192.168.0.174] 80 (http) open  
(UNKNOWN) [192.168.0.174] 79 (finger) : Connection timed out  
(UNKNOWN) [192.168.0.174] 78 (?) : Connection timed out  
(UNKNOWN) [192.168.0.174] 77 (?) : Connection timed out  
(UNKNOWN) [192.168.0.174] 76 (?) : Connection timed out  
(UNKNOWN) [192.168.0.174] 75 (?) : Connection timed out  
sent 0, rcvd 0
```

Результат сканирования сообщает нам, что порт 80 открыт, в то время как соединения на портах 79-75 завершились по таймеру. На скриншоте ниже показан захват Wireshark этого сканирования:



В этом захвате Netcat отправил несколько TCP SYN пакетов на порты 80, 79, 78 и так далее до 75 порта соответственно. Из-за различных факторов, пакеты могут отображаться в Wireshark не по порядку. Обратите внимание, что сервер отправил пакет TCP SYN-ACK с порта 80 что указывает на то, что порт открыт. Другие порты не ответили аналогичным пакетом SYN-ACK поэтому можно сделать вывод, что они закрыты. И наконец, Netcat закрыл это соединение отправив пакет FIN-ACK.

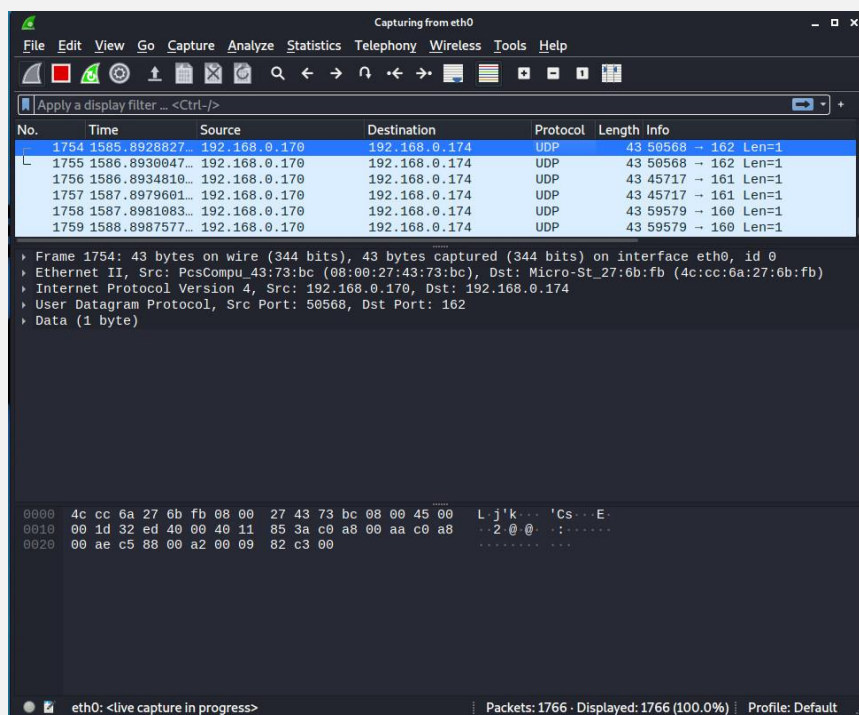
Сканирование UDP

Поскольку UDP не статичен и не предполагает трехстороннего рукопожатия, механизм, лежащий в основе UDP сканирования портов отличается от TCP. Давайте запустим сканирование портов UDP для портов 160-162. Это делается с помощью единственной опции Netcat, которую мы еще не использовали, -u, которая указывает на UDP сканирование:

```
kali@kali:~$ nc -nv -u -z -w 1 192.168.0.174 160-162
```

```
(kali@kali)-[~]
$ nc -nv -u -z -w 1 192.168.0.174 160-162
(UNKNOWN) [192.168.0.174] 162 (snmp-trap) open
(UNKNOWN) [192.168.0.174] 161 (snmp) open
(UNKNOWN) [192.168.0.174] 160 (?) open
```

Из захвата Wireshark видно, что сканирование UDP использует другой механизм, чем TCP сканирование:



Из захвата можно понять, что пустой UDP-пакет отправляется на определенный порт. Если порт назначения UDP открыт, пакет будет передан на прикладной уровень и ответ будет зависеть от того, как приложение запрограммировано реагировать на пустые пакеты. В данном примере приложение не посылает никакого ответа. Однако, если UDP порт назначения закрыт, то цель должна ответить сообщением "ICMP port unreachable". Большинство сканеров UDP используют стандартное сообщение "ICMP port unreachable", чтобы определить статус целевого порта. Однако этот метод может быть совершенно ненадежным, если целевой порт фильтруется брандмауэром. Фактически, в таких случаях сканер будет сообщать, что целевой порт открыт из-за отсутствия ICMP-сообщения.

Распространенные ошибки сканирования портов

Сканирование UDP может быть проблематичным по нескольким причинам. Во-первых, сканирование UDP часто ненадежно, поскольку брандмауэры и маршрутизаторы могут отбрасывать ICMP-пакеты. Это может привести к ложным срабатываниям и отображению портов как открытые, когда на самом деле они закрыты. Во-вторых, многие сканеры портов не сканируют все доступные порты, и обычно имеют заранее установленный список портов, которые сканируются. Это означает, что открытые порты UDP могут остаться незамеченными. Использование сканера портов UDP, специфичного для конкретного протокола, может помочь в получении более точных

результатов. И наконец, специалисты по тестированию на проникновение часто забывают сканировать открытые UDP-порты, вместо этого сосредотачиваясь на TCP-портах. Хотя и сканирование UDP может быть ненадежным, существует множество векторов атак, скрывающихся за открытыми UDP-портами.

Сканирование портов с помощью Nmap

[Nmap](#) – один из самых популярных, универсальных и надежных сканеров портов. Он активно разрабатывается уже более десяти лет и имеет множество функций, помимо сканирования портов.

Давайте рассмотрим несколько примеров сканирования портов, чтобы лучше понять Nmap и его возможности.

Стандартное Nmap TCP-сканирование сканирует 1000 наиболее популярных портов. Прежде чем мы начнем выполнять сканирование вслепую, давайте изучим количество трафика, отправляемого этим типом сканирования. Мы просканируем машину под управлением Windows, одновременно отслеживая количество трафика, отправленного на целевой узел с помощью iptables. Мы будем использовать несколько опций iptables. Во-первых, мы будем использовать опцию -I для вставки нового правила в заданную цепочку, которая в данном случае включает цепочки INPUT (входящий) и OUTPUT (исходящий), за которыми следует номер правила. Мы будем использовать -s для указания IP-адреса источника, -d для указания IP-адреса назначения и -j для ACCEPT (принятия) трафика. И наконец, мы используем опцию -Z, чтобы обнулить счетчики пакетов и байтов во всех цепочках.

Давайте выполним эти команды:

```
kali@kali:~$ sudo iptables -I INPUT 1 -s 192.168.0.174 -j ACCEPT
kali@kali:~$ sudo iptables -I OUTPUT 1 -d 192.168.0.174 -j ACCEPT
kali@kali:~$ sudo iptables -Z
```

Теперь давайте сгенерируем немного трафика с помощью Nmap:

```
kali@kali:~$ nmap 192.168.0.174
```

```
(kali@kali)-[~]
$ sudo iptables -I INPUT 1 -s 192.168.0.174 -j ACCEPT
[sudo] password for kali:

(kali@kali)-[~]
$ sudo iptables -I OUTPUT 1 -d 192.168.0.174 -j ACCEPT

(kali@kali)-[~]
$ sudo iptables -Z

(kali@kali)-[~]
$ nmap 192.168.0.174
Starting Nmap 7.92 ( https://nmap.org ) at 2022-07-12 10:43 EDT
Nmap scan report for 192.168.0.174
Host is up (0.0045s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
1688/tcp  open  nsjtp-data
7070/tcp  open  realserver

Nmap done: 1 IP address (1 host up) scanned in 4.95 seconds
```

Сканирование завершилось и выявило несколько открытых портов. Теперь давайте посмотрим на статистику iptables, чтобы получить представление о том, сколько трафика генерировало наше сканирование. Мы будем использовать опцию -v, чтобы добавить некоторую многословность в наш вывод, -n для включения числового вывода и -L для вывода списка правил, присутствующих во всех цепочках:

```
kali@kali:~$ sudo iptables -vn -L
```

```
(kali@kali)-[~]
$ sudo iptables -vn -L
Chain INPUT (policy ACCEPT 8 packets, 957 bytes)
 pkts bytes target    prot opt in     out     source            destination
  33  3998 ACCEPT     all  --  *      *       192.168.0.174     0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain OUTPUT (policy ACCEPT 1 packets, 72 bytes)
 pkts bytes target    prot opt in     out     source            destination
 2024 121K ACCEPT     all  --  *      *       0.0.0.0/0         192.168.0.174
```

Согласно этим данным, сканирование 1000 портов сгенерировало около 121 КБ трафика. Давайте воспользуемся командой iptables -Z, чтобы снова обнулить счетчики пакетов и байтов во всех цепочках, и запустим еще одно сканирование с использованием опции -p для указания всех TCP-портов:

```
kali@kali:~$ sudo iptables -Z
kali@kali:~$ nmap -p 1-65535 192.168.0.174
```

```
└─$ nmap -p 1-65535 192.168.0.174
Starting Nmap 7.92 ( https://nmap.org ) at 2022-07-12 11:02 EDT
Nmap scan report for 192.168.0.174
Host is up (0.0036s latency).
Not shown: 65527 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
1540/tcp  open  rds
1688/tcp  open  nsjtp-data
7070/tcp  open  realserver

Nmap done: 1 IP address (1 host up) scanned in 175.67 seconds

(kali@kali)-[~]
└─$ sudo iptables -vn -L
Chain INPUT (policy ACCEPT 3 packets, 142 bytes)
 pkts bytes target    prot opt in     out     source destination
  193 14500 ACCEPT    all  --  *      *       192.168.0.174 0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source destination
 134K 8008K ACCEPT    all  --  *      *       0.0.0.0/0      192.168.0.174
```

Аналогичное локальное сканирование портов, явно проверяющее все 65535 портов, сгенерировало около 8 МБ трафика, что значительно больше. Приведенные выше результаты означают, что полное сканирование Nmap сети класса C (254 хоста) приведет к отправке в сеть более 1000 МБ трафика. В идеальной ситуации полное сканирование TCP и UDP портов каждой отдельной целевой машины дало бы наиболее точную информацию об открытых сетевых службах.

Далее мы рассмотрим некоторые из различных методов сканирования Nmap.

"Стелс" или SYN-сканирование

Предпочтительной техникой сканирования в Nmap является SYN, или "стелс" сканирование. Использование SYN сканирования имеет много преимуществ, поэтому оно используется по умолчанию, когда в команде nmap не указана техника сканирования, а пользователь имеет необходимые привилегии для работы с ["сырыми сокетами"](#).

SYN-сканирование – это метод сканирования портов TCP, который заключается в отправке SYN-пакетов на различные порты целевой машины без завершения рукопожатия. Если порт TCP открыт, с целевой машины должен быть отправлен SYN-ACK, информирующий нас о том, что порт открыт. В этот момент сканер портов не утруждает себя отправкой последнего ACK для завершения трехстороннего рукопожатия.

```
kali@kali:~$ sudo nmap -sS 192.168.0.174
```


Поскольку трехстороннее рукопожатие не завершается, информация не передается на прикладной уровень и, как следствие, не появляется в журналах приложений. Сканирование SYN также быстрее и эффективнее, поскольку отправляется и принимается меньше пакетов.

P.S. Обратите внимание, что термин "стелс" относится к тому факту, что в прошлом примитивные брандмауэры не регистрировали незавершенные TCP-соединения. В современных брандмауэрах это уже не так, и даже если термин "стелс" остался, он может вводить в заблуждение.

Сканирование TCP Connect

Если пользователь, запускающий Nmap, не имеет привилегий для работы с сырыми сокетами, Nmap будет по умолчанию использовать технику сканирования TCP Connect, описанную ранее. Поскольку при сканировании Nmap TCP Connect используется API Berkeley Sockets для выполнения трехстороннего рукопожатия, оно не требует повышенных привилегий. Однако, поскольку Nmap должен дожидаться завершения соединения, прежде чем API вернет статус соединения, сканирование соединения занимает гораздо больше времени, чем сканирование SYN. Бывают случаи, когда нам нужно специально выполнить Connect сканирование с помощью Nmap, например, при сканировании через определенные типы прокси. Мы используем опцию -sT для запуска Connect сканирования:

```
kali@kali:~$ nmap -sT 192.168.0.174
```

UDP сканирование с помощью Nmap

При выполнении сканирования UDP Nmap будет использовать комбинацию двух различных методов для определения того, открыт или закрыт порт. Для большинства портов он использует стандартный метод "ICMP port unreachable", описанный ранее, посылая пустой пакет на данный порт. Однако для распространенных портов, таких как порт 161, который используется SNMP, он отправит специфичный для протокола SNMP пакет в попытке получить ответ от приложения, привязанного к этому порту. Для выполнения сканирования UDP используется опция -sU, а для доступа к сырым сокетам требуется использование sudo:

```
kali@kali:~$ sudo nmap -sU 192.168.0.174
```

```
(kali@kali)-[~]  
$ sudo nmap -sU 192.168.0.174  
[sudo] password for kali:  
Starting Nmap 7.92 ( https://nmap.org ) at 2022-07-12 11:52 EDT  
Nmap scan report for 192.168.0.174  
Host is up (0.0090s latency).  
Not shown: 999 open|filtered udp ports (no-response)  
PORT      STATE SERVICE  
137/udp   open  netbios-ns  
MAC Address: [REDACTED] (Micro-star Intl)
```

Сканирование UDP (-sU) также может быть использовано в сочетании с опцией сканирования TCP SYN (-sS) для получения более полной картины о нашей цели:

```
kali@kali:~$ sudo nmap -sU -sS 192.168.0.174
```

Сетевой "свипинг"

Чтобы справиться с большим количеством хостов или попытаться сэкономить сетевой трафик, мы можем попытаться исследовать цели, используя технику сетевого сканирования, при которой мы начинаем с широкого сканирования и используем более специфическое сканирование против интересующих нас хостов. При выполнении сетевого сканирования с помощью Nmap с опцией -sn процесс обнаружения хоста состоит не только из отправки эхо-запроса ICMP. Nmap также отправляет пакет TCP SYN на порт 443, пакет TCP ACK на порт 80 и запрос ICMP timestamp, чтобы проверить, доступен хост или нет.

```
kali@kali:~$ nmap -sn 192.168.0.1-254
```

```
(kali@kali)-[~]  
$ nmap -sn 192.168.0.1-254  
Starting Nmap 7.92 ( https://nmap.org ) at 2022-07-12 12:19 EDT  
Nmap scan report for dlinkrouter.Dlink (192.168.0.1)  
Host is up (0.0036s latency).  
Nmap scan report for MBP-ADMIN.Dlink (192.168.0.130)  
Host is up (0.00096s latency).  
Nmap scan report for kali.Dlink (192.168.0.141)  
Host is up (0.0062s latency).  
Nmap scan report for 192.168.0.174  
Host is up (0.0035s latency).  
Nmap done: 254 IP addresses (4 hosts up) scanned in 6.86 seconds
```

Поиск "живых" машин с помощью команды `grep` в стандартном выводе Nmap может быть громоздким. Вместо этого давайте воспользуемся параметром вывода `-oG`, чтобы сохранить эти результаты в формате, которым легче пользоваться:

```
kali@kali:~$ nmap -v -sn 192.168.0.1-254 -oG ping.txt
```

```
kali@kali:~$ grep Up ping.txt | cut -d " " -f 2
```

```
(kali@kali)-[~]  
$ grep Up ping.txt | cut -d " " -f 2  
192.168.0.1  
192.168.0.130  
192.168.0.141  
192.168.0.174
```

Мы также можем проверить определенные TCP или UDP порты в сети, прощупывая общие службы и порты в попытке найти системы, которые могут быть полезны или имеют известные уязвимости. Такое сканирование, как правило, более точное, чем сканирование с помощью "ping sweep":

```
kali@kali:~$ nmap -p 80 192.168.0.1-254 -oG sweep.txt
```

```
(kali@kali)-[~]  
$ nmap -p 80 192.168.0.1-254 -oG sweep.txt  
Starting Nmap 7.92 ( https://nmap.org ) at 2022-07-12 12:35 EDT  
Nmap scan report for dlinkrouter.Dlink (192.168.0.1)  
Host is up (0.0063s latency).  
  
PORT      STATE SERVICE  
80/tcp    open  http  
  
Nmap scan report for MBP-ADMIN.Dlink (192.168.0.130)  
Host is up (0.00046s latency).  
  
PORT      STATE SERVICE  
80/tcp    closed http  
  
Nmap scan report for kali.Dlink (192.168.0.141)  
Host is up (0.0031s latency).  
  
PORT      STATE SERVICE  
80/tcp    closed http  
  
Nmap scan report for 192.168.0.174  
Host is up (0.0027s latency).  
  
PORT      STATE SERVICE  
80/tcp    open  http
```

Для экономии времени и сетевых ресурсов мы также можем сканировать несколько IP-адресов, проверяя короткий список общих портов. Например, давайте проведем сканирование на двадцать TCP-портов с помощью опции --top-ports и включим определение версии ОС с помощью опции -A:

```
kali@kali:~$ nmap -sT -A --top-ports=20 192.168.0.1-254 -oG top-port.txt
```

```
Nmap scan report for 192.168.0.174
Host is up (0.0032s latency).

PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          Microsoft ftpd
| ftp-syst:
|_  SYST: Windows_NT
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| 12-20-21 07:22PM <DIR> 1
| 06-07-22 12:24AM <DIR> aspNet_client
22/tcp    filtered ssh
23/tcp    filtered telnet
25/tcp    filtered smtp
53/tcp    filtered domain
80/tcp    open  http         Microsoft IIS httpd 10.0
|_ http-server-header: Microsoft-IIS/10.0
|_ http-title: IIS Windows
|_ http-methods:
|_  Potentially risky methods: TRACE
110/tcp   filtered pop3
111/tcp   filtered rpcbind
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
143/tcp   filtered imap
443/tcp   filtered https
445/tcp   open  microsoft-ds Windows 10 Pro 15063 microsoft-ds (workgroup: WORKGROUP)
| fingerprint-strings:
|_  SMBProgNeg:
|_  SMBv1:
|_  Jm0rH
```

Двадцать самых популярных портов, которые определяет Nmap, берутся из файла ***/usr/share/nmap/nmap-services***. Файл использует простой формат из трех столбцов, разделенных пробелами. Он содержит список портов, протоколов (TCP/UDP) и служб, которые обычно работают на этих портах. Этот файл позволяет Nmap более точно определять, какие службы запущены на целевых хостах, и устанавливать приоритет сканирования наиболее популярных портов.

Каждая строка в этом файле включает:

- Номер порта и протокол.
- Имя службы, которая обычно использует этот порт.
- Статистический рейтинг популярности порта, что помогает Nmap определять, какие порты чаще всего сканировать в первую очередь.

Все, что находится после третьего столбца, игнорируется, но обычно используется для комментариев, что видно по использованию знака (#). Частота порта основана на том, как часто порт был обнаружен открытым во время исследований Интернета.

```
kali@kali:~$ cat /usr/share/nmap/nmap-services
```

```
systat 11/tcp 0.000075 # Active Users
systat 11/udp 0.000577 # Active Users
unknown 12/tcp 0.000063
daytime 13/tcp 0.003927
daytime 13/udp 0.004827
unknown 14/tcp 0.000038
netstat 15/tcp 0.000038
unknown 16/tcp 0.000050
qotd 17/tcp 0.002346 # Quote of the Day
qotd 17/udp 0.009209 # Quote of the Day
msp 18/tcp 0.000000 # Message Send Protocol | Message Send Protocol (historic)
msp 18/udp 0.000610 # Message Send Protocol
chargen 19/tcp 0.002559 # ttytst source Character Generator | Character Generator
chargen 19/udp 0.015865 # ttytst source Character Generator
ftp-data 20/sctp 0.000000 # File Transfer [Default Data] | FTP
ftp-data 20/tcp 0.001079 # File Transfer [Default Data]
ftp-data 20/udp 0.001878 # File Transfer [Default Data]
ftp 21/sctp 0.000000 # File Transfer [Control] | File Transfer Protocol [Control]
ftp 21/tcp 0.197667 # File Transfer [Control]
ftp 21/udp 0.004844 # File Transfer [Control]
ssh 22/sctp 0.000000 # Secure Shell Login | The Secure Shell (SSH) Protocol
ssh 22/tcp 0.182286 # Secure Shell Login
ssh 22/udp 0.003905 # Secure Shell Login
telnet 23/tcp 0.221265
telnet 23/udp 0.006211
priv-mail 24/tcp 0.001154 # any private mail system
priv-mail 24/udp 0.000329 # any private mail system
smtp 25/tcp 0.131314 # Simple Mail Transfer
smtp 25/udp 0.001285 # Simple Mail Transfer
rsftp 26/tcp 0.007991 # RSFTP
nsw-fe 27/tcp 0.000138 # NSW User System FE
```

На этом этапе мы уже можем провести более полное сканирование отдельных машин, которые богаты различными сервисами или представляют интерес по другим причинам. Существует множество различных способов (творческих подходов) к сканированию которые заслуживают дальнейшего изучения.

OS Fingerprinting

Nmap имеет встроенную функцию под названием OS Fingerprinting, которую можно включить с помощью опции `-O`. Эта функция пытается определить операционную систему цели путем анализа возвращаемых пакетов. Это возможно, потому что операционные системы часто имеют немного разные реализации стека TCP/IP (например, разные значения [TTL](#) и т.д.), и эти небольшие различия создают отпечаток, который Nmap может определить. Nmap проверит трафик, полученный с целевой машины, и попытается сопоставить отпечатки с уже известным списком. Для примера рассмотрим простое сканирование ОС отпечатков:

```
kali@kali:~$ sudo nmap -O 192.168.0.174
```



```
(kali@kali)-[~]
$ sudo nmap -O 192.168.0.174
[sudo] password for kali:
Starting Nmap 7.92 ( https://nmap.org ) at 2022-07-12 13:09 EDT
Nmap scan report for 192.168.0.174
Host is up (0.0020s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
1688/tcp  open  nsjtp-data
7070/tcp  open  realserver
MAC Address: [REDACTED] (Micro-star Intl)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose phone
Running (JUST GUESSING): Microsoft Windows 10|2008|7|Phone (92%), FreeBSD 6.X (90%)
OS CPE: cpe:/o:microsoft:windows_10 cpe:/o:freebsd:freebsd:6.2 cpe:/o:microsoft:windows_server_2008::sp1 cpe:/o:microsoft:windows_7 cpe:/o:microsoft:windows
Aggressive OS guesses: Microsoft Windows 10 (92%), FreeBSD 6.2-RELEASE (90%), Microsoft Windows Server 2008 SP1 (87%), Microsoft Windows 7 (87%), Microsoft Windows 10 1703 (86%), Microsoft Windows 10 1511 - 1607 (86%), Microsoft Windows Phone 7.5 or 8.0 (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.47 seconds
```

Ответ предполагает, что операционной системой этой цели является Windows 10 с вероятностью в 92%. И это правильный ответ :)

Обратите внимание, что отпечатки пальцев ОС не всегда точны на 100%, а являются лишь предположением. Для подтверждения результатов сканирования отпечатков ОС следует более тщательно изучить цель.

Захват баннера / Эnumерация служб

Мы также можем определить службы, работающие на определенных портах, путем просмотра баннеров служб (-sV) и запуска различных скриптов перечисления ОС и служб (-A) на целевом хосте.

```
kali@kali:~$ nmap -sV -sT -A 192.168.0.174
```

```
(kali@kali)-[~]
$ nmap -sV -sT -A 192.168.0.174
Starting Nmap 7.91 ( https://nmap.org ) at 2022-07-12 16:58 EDT
Nmap scan report for 192.168.0.174
Host is up (0.00093s latency).
Not shown: 993 filtered ports
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          Microsoft ftpd
ftp-anon: Anonymous FTP login allowed (FTP code 230)
12-20-21  07:22PM    <DIR>        1
_06-07-22 12:24AM    <DIR>        aspnet_client
ftp-syst:
_ SYST: Windows_NT
80/tcp    open  http         Microsoft IIS httpd 10.0
_http-methods:
_ Potentially risky methods: TRACE
_http-server-header: Microsoft-IIS/10.0
_http-title: IIS Windows
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Windows 10 Pro 15063 microsoft-ds (workgroup: WORKGROUP)
fingerprint-strings:
_SMBProgNeg:
_SMB:
_ \x93*=2
1688/tcp  open  msrpc        Microsoft Windows RPC
2869/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
_http-server-header: Microsoft-HTTPAPI/2.0
_http-title: Service Unavailable
Service Info: Host: DESKTOP-JA7CNCQ; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Помните, что баннеры могут быть изменены системными администраторами. В связи с этим они могут быть намеренно установлены на поддельные имена служб, чтобы ввести в заблуждение потенциального злоумышленника.

Захват баннеров оказывает значительное влияние на количество используемого трафика, а также на скорость сканирования. Мы всегда должны помнить об опциях, которые мы используем в Nmap, и о том, как они влияют на наше сканирование.

Nmap Scripting Engine (NSE)

Мы можем использовать Nmap Scripting Engine (NSE) для запуска созданных пользователем сценариев, чтобы автоматизировать различные задачи сканирования. Эти сценарии выполняют широкий спектр функций, включая DNS перечисление, атаки типа брутфорс и даже выявление уязвимостей. Сценарии NSE расположены в каталоге /usr/share/nmap/scripts. Например, сценарий smb-os-discovery пытается подключиться к службе SMB на целевой машине и определить ее операционную систему.

```
kali@kali:~$ nmap 192.168.0.174 --script=smb-os-discovery
```

```
(kali@kali)-[~]
$ nmap 192.168.0.174 --script=smb-os-discovery
Starting Nmap 7.91 ( https://nmap.org ) at 2022-07-12 17:09 EDT
Nmap scan report for 192.168.0.174
Host is up (0.00056s latency).
Not shown: 994 filtered ports
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
1688/tcp  open  nsjtp-data

Host script results:
| smb-os-discovery:
|_ OS: Windows 10 Pro 15063 (Windows 10 Pro 6.3)
|   OS CPE: cpe:/o:microsoft:windows_10::-
|   Computer name: DESKTOP-JA7CNCQ
|   NetBIOS computer name: DESKTOP-JA7CNCQ\x00
|   Workgroup: WORKGROUP\x00
|_  System time: 2022-07-13T00:09:12+03:00

Nmap done: 1 IP address (1 host up) scanned in 4.84 seconds
```

Скрипт Nmap Scripting Engine (NSE) под названием dns-zone-transfer проверяет, открыт ли сервер для зонного трансфера. Это может быть полезно при тестировании безопасности, так как неправильно настроенный сервер может предоставить полную информацию о домене, что является уязвимостью и может использоваться злоумышленниками для получения информации о внутренней структуре сети.

Выполняя этот скрипт, Nmap отправляет запрос на зонный трансфер к DNS-серверу и, в случае успешного выполнения, возвращает всю доступную информацию о домене. Это помогает тестировать, правильно ли настроен сервер для защиты от нежелательного доступа к DNS-записям:

```
kali@kali:~$ nmap --script=dns-zone-transfer -p 53 ns2.megacorpone.com
```

```

kali@kali: ~
File Actions Edit View Help

(kali@kali)-[~]
$ nmap --script=dns-zone-transfer -p 53 ns2.megacorpone.com
Starting Nmap 7.91 ( https://nmap.org ) at 2022-07-12 17:11 EDT
Nmap scan report for ns2.megacorpone.com (51.222.39.63)
Host is up (0.14s latency).

PORT      STATE SERVICE
53/tcp    open  domain
| dns-zone-transfer:
|   megacorpone.com.      SOA ns1.megacorpone.com. admin.megacorpone.com.
|   megacorpone.com.      TXT "Try Harder"
|   megacorpone.com.      TXT "google-site-verification=U7B_b0HNeBtY4qYGQ
|   ZNsEYXfCJ32hMNV3GtC0wWq5pA"
|   megacorpone.com.      MX 10 fb.mail.gandi.net.
|   megacorpone.com.      MX 20 spool.mail.gandi.net.
|   megacorpone.com.      MX 50 mail.megacorpone.com.
|   megacorpone.com.      MX 60 mail2.megacorpone.com.
|   megacorpone.com.      NS ns1.megacorpone.com.
|   megacorpone.com.      NS ns2.megacorpone.com.
|   megacorpone.com.      NS ns3.megacorpone.com.
|   admin.megacorpone.com. A 51.222.169.208
|   beta.megacorpone.com.  A 51.222.169.209
|   fs1.megacorpone.com.   A 51.222.169.210
|   intranet.megacorpone.com. A 51.222.169.211
|   mail.megacorpone.com.  A 51.222.169.212
|   mail2.megacorpone.com. A 51.222.169.213
|   ns1.megacorpone.com.   A 51.79.37.18
|   ns2.megacorpone.com.   A 51.222.39.63
|   ns3.megacorpone.com.   A 66.70.207.180
|   router.megacorpone.com. A 51.222.169.214
|   siem.megacorpone.com.  A 51.222.169.215
|   snmp.megacorpone.com.  A 51.222.169.216
|   support.megacorpone.com. A 51.222.169.218
|   syslog.megacorpone.com. A 51.222.169.217
|   test.megacorpone.com.  A 51.222.169.219
|   vpn.megacorpone.com.   A 51.222.169.220
|   www.megacorpone.com.   A 149.56.244.87
|   www2.megacorpone.com.  A 149.56.244.87
|   _megacorpone.com.      SOA ns1.megacorpone.com. admin.megacorpone.com.
Nmap done: 1 IP address (1 host up) scanned in 5.56 seconds

```

Для просмотра дополнительной информации о скрипте мы можем использовать опцию `--script-help`, которая отображает описание скрипта и URL, где можно найти более подробную информацию, например, аргументы скрипта и примеры использования.

```
kali@kali:~$ nmap --script-help dns-zone-transfer
```

```

(kali@kali)-[~]
$ nmap --script-help dns-zone-transfer 255 x
Starting Nmap 7.91 ( https://nmap.org ) at 2022-07-12 17:15 EDT

dns-zone-transfer
Categories: intrusive discovery
https://nmap.org/nsedoc/scripts/dns-zone-transfer.html
Requests a zone transfer (AXFR) from a DNS server.

The script sends an AXFR query to a DNS server. The domain to query is
determined by examining the name given on the command line, the DNS
server's hostname, or it can be specified with the
<code>dns-zone-transfer.domain</code> script argument. If the query is
successful all domains and domain types are returned along with common
type specific data (SOA/MX/NS/PTR/A).

This script can run at different phases of an Nmap scan:
* Script Pre-scanning: in this phase the script will run before any
Nmap scan and use the defined DNS server in the arguments. The script
arguments in this phase are: <code>dns-zone-transfer.server</code> the
DNS server to use, can be a hostname or an IP address and must be
specified. The <code>dns-zone-transfer.port</code> argument is optional
and can be used to specify the DNS server port.
* Script scanning: in this phase the script will run after the other
Nmap phases and against an Nmap discovered DNS server. If we don't
have the "true" hostname for the DNS server we cannot determine a
likely zone to perform the transfer on.

Useful resources
* DNS for rocket scientists: http://www.zytrax.com/books/dns/
* How the AXFR protocol works: http://cr.yp.to/djbdns/axfr-notes.html

```

Для тех случаев, когда доступ в Интернет недоступен, большую часть этой информации можно также найти в самом файле сценария NSE. Уделите время изучению различных сценариев NSE, так как многие из них полезны и экономят время.

Команды, используемые в уроке:

```
kali@kali:~$ nc -nv -w 1 -z 192.168.0.174 75-80
```

```
kali@kali:~$ nc -nv -u -z -w 1 192.168.0.174 160-162
```

```
kali@kali:~$ sudo iptables -I INPUT 1 -s 192.168.0.174 -j ACCEPT
```

```
kali@kali:~$ sudo iptables -I OUTPUT 1 -d 192.168.0.174 -j ACCEPT
```

```
kali@kali:~$ sudo iptables -Z
```

```
kali@kali:~$ nmap 192.168.0.174
```

```
kali@kali:~$ sudo iptables -vn -L
```

```
kali@kali:~$ nmap -p 1-65535 192.168.0.174
```

```
kali@kali:~$ sudo nmap -sS 192.168.0.174
```

```
kali@kali:~$ nmap -sT 192.168.0.174
```

```
kali@kali:~$ sudo nmap -sU 192.168.0.174
```

```
kali@kali:~$ sudo nmap -sU -sS 192.168.0.174
```

```
kali@kali:~$ nmap -sn 192.168.0.1-254
```

```
kali@kali:~$ nmap -v -sn 192.168.0.1-254 -oG ping.txt
```

```
kali@kali:~$ grep Up ping.txt | cut -d " " -f 2
```

```
kali@kali:~$ nmap -p 80 192.168.0.1-254 -oG sweep.txt
```

```
kali@kali:~$ nmap -sT -A --top-ports=20 192.168.0.1-254 -oG top-port.txt
```

```
kali@kali:~$ cat /usr/share/nmap/nmap-services
```

```
kali@kali:~$ sudo nmap -O 192.168.0.174
```



```
kali@kali:~$ nmap -sV -sT -A 192.168.0.174
```

```
kali@kali:~$ nmap 192.168.0.174 --script=smb-os-discovery
```

```
kali@kali:~$ nmap --script=dns-zone-transfer -p 53 ns2.megacorpone.com
```

```
kali@kali:~$ nmap --script-help dns-zone-transfer
```

Задание:

1. Используйте Nmap для проведения ping-сканирования целевого диапазона IP-адресов в вашей локальной сети и сохраните результаты в файл. Используйте gper, чтобы показать машины, которые находятся в сети.
2. Просканируйте IP-адреса, которые вы нашли в задании 1, на наличие открытых портов. Используйте Nmap, чтобы найти версию операционной системы для каждого IP-адреса.